

# Components

The benchmark dashboard journey, the comparison, the history, the setup guide and the vision panels

---

EdgePilot AI - Team 15

Documentation set, 26 August 2026

6 documents in this volume

## src/components/ - React components

**Work packages:** product UI dashboard, and vision benchmark.

### What this folder is

---

Everything the user actually sees, in two groups by feature.

### What happened here

---

Four files were removed from this level: `ErrorBoundary.tsx`, `Loading.tsx`, `LoadingStates.tsx` and `providers/`. Three of those were overlapping answers to the same question, and the App Router already has a convention for it - `loading.tsx` and `error.tsx` beside the route. What genuinely needed sharing is 85 lines in `dashboard/StateViews.tsx`.

`vision/` is new, and it holds a component with an unusual constraint: it must work entirely in the browser, because the images it handles are not allowed to reach the server.

### Subfolders

---

| Subfolder  | What it holds                                                                                                                                                                                 |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dashboard/ | 11 files - the benchmark journey                                                                                                                                                              |
| vision/    | Both vision run panels, the browser run store, and the threshold note                                                                                                                         |
| compare/   | The comparison application - the UI the comparison engine never had                                                                                                                           |
| history/   | Every run this session produced, and what may leave the machine                                                                                                                               |
| setup/     | SetupGuide - the /setup page: OLLAMA_ORIGINS instructions per OS with the site's real origin filled in, a "check the connection" probe, and the visitor's own cloud keys with a save-and-test |

### Connected folders

---

- `src/app/` - the routes that mount these.
- `docs/dashboard/` - the module guide.
- `tests/dashboard/` - the helper tests.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- `src/`

## src/components/compare/ - the comparison application

**Work package:** product UI and benchmark dashboard.

### What this folder is

---

One 899-line client component driving `/api/v1/comparisons`: pick two to four entrants, send one prompt, read a per-dimension verdict.

### What happened here

---

New, and it is the surface that made the comparison engine usable. Three things in it are worth knowing because they show it was written against this codebase rather than in spite of it.

**It mirrors the server's validation exactly** - 4,000-character prompt cap, 1 to 20 iterations, at most four entrants - which is the convention the dashboard docs state: client validation mirrors, never replaces.

**It understands the measurement design.** It tells the user "{iterations} measured + 1 discarded cold start" and "two local entrants run one after the other", rather than just calling the endpoint. Those are the N+1 warm-up and ComparisonPlanner's sequential rule, surfaced rather than hidden.

**It renders "no winner" as a result.** Every dimension carries an honesty code, and a row where the observed ranges overlap says so instead of showing a blank. That is the whole argument of ComparisonReport; a UI that rendered it as an empty cell would have quietly undone it.

It also refuses the same model twice, mirroring the planner's refusal - the labels are `provider + model` and the report's tally is keyed by them, so a duplicate collides onto one key and merges two entrants into one verdict.

**Hosted-mode pass.** Ollama entrants are measured in the browser, one after another, before the request is sent, with the progress line naming the model being measured; cloud entrants are measured by the server. The model field is a list for every provider - the visitor's installed models for Ollama, the vendor's list for Gemini and Groq - and the placeholder cannot be submitted. The second entrant defaults to the first configured cloud provider (Gemini before Groq) so the page opens as the local-versus-cloud pairing the app exists for, and never overrides a choice already made.

### Files

---

| File           | What it is                                                                                          |
|----------------|-----------------------------------------------------------------------------------------------------|
| CompareApp.tsx | 899 lines. Entrant form, egress preview, plan, per-dimension verdicts, and the session-log controls |

---

### Connected folders

---

- `src/app/compare/` - the route.
- `src/app/api/v1/comparisons/` - the endpoint.
- `src/modules/benchmark/core/services/` - ComparisonPlanner, ComparisonReport.
- `src/components/vision/` - `runHistory.ts`, where a completed comparison is stored.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- `src/components/`

## src/components/dashboard/ - the benchmark journey

**Work package:** product UI and benchmark dashboard / integration lead.

### What this folder is

Eleven files implementing the four-step user journey: workload, provider and model, run, results. This is where the 692 lines that used to be in `app/dashboard/page.tsx` now live, and the move is what makes them testable.

### What happened here

The largest change in the folder is a **deletion**. `SetupPanel.tsx` went from 322 lines to 201 - 379 lines differing - because the UUID field and everything around it went away. Users were being asked to paste a workload uuid by hand, because the endpoint that should have created one did not persist. Once `POST /api/v1/workloads` wrote real rows, the field, the placeholder-id detection and the "this was not saved" warning all became dead weight.

The identity that replaced it is `session.ts`, and the important property is that it is **never rendered**. A user should no more be asked to see their own row-ownership key than to type their own cookie.

`InstalledModels.tsx` and `VisionHandoff.tsx` are new. The installed-models panel is always visible and filters by step: on step 1 models that cannot serve the chosen task are greyed out; from step 2 only models that will work are offered at all.

The input and output format fields became read-only derived text - an informant, not a control. They describe what the chosen task sends and returns; they were never things a user could meaningfully change.

**Hosted-mode pass.** `InstalledModels`, `ProviderPanel` and `DashboardApp` ask the visitor's own Ollama from the browser instead of the server; `ProviderPanel` lists Gemini and Groq models from the vendors (`getProviderModels`) rather than a free-text box, and every placeholder option is disabled; `DashboardApp` clears the model when the provider changes so a Groq name is never submitted to Gemini; `RunPanel` says where the run happens. `apiKeys.ts` is new, and `api.ts` attaches the visitor's keys to every call and, for Ollama, measures in the tab before posting the recorded result.

### Files

| File                             | What it is                                                                                                                                                                                                                 |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DashboardApp.tsx</code>    | 204 lines. The step machine and shared state                                                                                                                                                                               |
| <code>SetupPanel.tsx</code>      | 202 lines. Step 1 - the workload. No uuid field any more                                                                                                                                                                   |
| <code>ProviderPanel.tsx</code>   | 295 lines. Step 2 - the live catalogue. Unconfigured providers are visible but not selectable                                                                                                                              |
| <code>RunPanel.tsx</code>        | 316 lines. Step 3 - prompt, iterations, elapsed feedback. Tells the user their iteration count will be raised by one for the discarded cold start                                                                          |
| <code>RunResults.tsx</code>      | 414 lines. Step 4 - per-iteration table with provenance badges, null-honest summary tiles, the fallback chain, readiness and its stored breakdown, evidence, assumptions, limitations, JSON export, and the cold-start row |
| <code>InstalledModels.tsx</code> | 245 lines. The always-visible model panel, with <code>fitFor</code> deciding what is offered at each step                                                                                                                  |
| <code>VisionHandoff.tsx</code>   | 88 lines. The route into the vision dashboard                                                                                                                                                                              |
| <code>StateViews.tsx</code>      | 85 lines. <code>EmptyState</code> , <code>ErrorState</code> , <code>LoadingState</code> - what replaced three separate top-level components                                                                                |
| <code>apiKeys.ts</code>          | The visitor's own cloud keys, in this browser's <code>localStorage</code> ; <code>apiKeyHeaders()</code> attaches them to every API call. Components get only "is one set", never the value                                |

| File       | What it is                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| api.ts     | The one typed gateway. For Ollama, getLocalRuntime and runBenchmark/runComparison talk to the visitor's own runtime from the tab (see infrastructure/browser-ollama.ts) and send the measurements to the server to be scored - the server cannot reach a visitor's machine once hosted. Every request goes through it, so components render exactly one error shape. Types come from the benchmark module's DTOs via import type, erased at build time, so the dashboard cannot drift from the measurement envelope without a compile error |
| format.ts  | 86 lines. Display helpers whose job is that <b>no component ever turns a null into a 0</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| session.ts | 92 lines. The browser's own durable id in localStorage, sent as x-edgepilot-session, never displayed. localStorage and not sessionStorage, because a workload registered in one tab has to still be yours in the next tab, and tomorrow                                                                                                                                                                                                                                                                                                     |

## Connected folders

- src/app/dashboard/ - the route and the .epd-scoped stylesheet.
- src/app/api/v1/ - what api.ts talks to.
- src/lib/ - sessionOwner.ts, the server half of session.ts.
- src/modules/benchmark/application/dtos/ - the types api.ts imports.
- tests/dashboard/ - format.test.ts.
- docs/dashboard/ - the module guide, including the requests this folder made of the other work packages.

## Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/local-runtime/
- src/app/api/v1/providers/
- src/components/setup/
- src/app/api/v1/readiness/[id]/ (FOLDER.md)
- src/app/api/v1/session-log/
- src/app/api/v1/workloads/
- src/app/vision-benchmark/
- src/components/
- src/components/history/
- src/core/logging/

## src/components/history/ - everything this session produced

**Work package:** shared.

### What this folder is

---

One client component rendering three run tables and the session-log controls.

### What happened here

---

New. Three tables, one per kind of run - text and code, comparisons, vision - each with per-row download, download-all, select-some, and clear. Plus the activity log: download it, or share it through the existing consent flow.

Two rules it exists to hold:

**The preview is the payload.** The share section renders `would_send` directly, and the consent statement and disclosure lines come from the server rather than being retyped here. A preview that described the upload instead of being it would drift, and a drifted preview is worse than none.

**A dataset you supplied is download-only.** Runs against the built-in dataset may be shared; runs against your own images may not, because their sample ids are your filenames, their labels are your folder names, and the outputs are a model describing your pictures. The table marks every row `built-in` or `yours` so this is visible before anything is clicked.

The session id is never rendered, the same rule `components/dashboard/session.ts` states.

**Hosted-mode pass.** The database panel that sat at the bottom of the page was removed with its endpoint - a debugging aid on a user-facing page, and one that told any visitor there was a Postgres to talk to. The vision-runs block was brought in line with the other two (per-row download, download-all, clear; no checkbox selection). The text-and-code table gained a **Measured in** column: this browser for an Ollama run the tab made against the visitor's own machine, server for a cloud run.

### Files

---

| File                            | What it is                                                                                                   |
|---------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>SessionHistory.tsx</code> | Three run tables, each with per-row and bulk download and clear, the log export, and the consent-gated share |

---

### Connected folders

---

- `src/app/history/` - the route and its header.
- `src/components/vision/` - `runHistory.ts`, the store all three tables read.
- `src/components/dashboard/` - `api.ts` supplies the share preview and upload functions.
- `src/core/logging/` - what the activity log is.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- `src/components/`

## src/components/setup/ - the setup guide

**Work package:** product UI and benchmark dashboard.

### What this folder is

---

One client component, SetupGuide, mounted by /setup.

### What happened here

---

Two sections. **Ollama:** the operating system is guessed from the user agent so the right tab opens, the site's real origin is read from `window.location` and written into the command, and a button probes the visitor's Ollama from the tab and reports "connected, N models" or the remedy. A passed check is remembered in that browser, so a returning visitor is told there is nothing to redo. On a page served from localhost the section says so and skips the steps.

**Keys:** two password fields with show/hide, Save, and Save & test - the test lists the vendor's models with that key, so a wrong key is caught before a run spends it. Values go to `localStorage` through `apiKeys.ts`; nothing that renders is ever given the key itself.

The command shown uses the site's own origin, not \*. Allowing every website would hand the visitor's GPU to any page they open; the note beneath says how to widen it if they want to.

### Files

---

| File           | What it is                                                                               |
|----------------|------------------------------------------------------------------------------------------|
| SetupGuide.tsx | The page body: OS tabs and per-OS commands, the connection check, and the two key fields |

### Connected folders

---

- `src/app/setup/` - the route.
- `src/components/dashboard/` - `apiKeys.ts` and `api.ts`, where the keys are stored and attached.

### Referenced from

---

These folders point here. Each link below resolves in both directions.

- `src/components/`

## src/components/vision/ - bring your own dataset

**Work package:** vision benchmark.

### What this folder is

Four files. One component takes your own images and never lets them leave the machine; another runs the dataset the project ships with; the other two are the store behind them and the note that says what "passed" means.

### What happened here

New. A user picks a folder laid out one directory per class (`webkitdirectory`), and the component measures a real model against their own photographs without a single image reaching the EdgePilot server. Resizing and hashing happen in the tab; classification goes from the tab straight to the user's own Ollama on localhost.

That is not a convenience decision. Benchmarking confidential data is this project's stated non-goal, and sending images to a server to be resized would have contradicted the warning the rest of the application gives.

Four things had to be got right, and three of them were got wrong first:

- **labelOf requires three path parts.** With two, a loose file sitting at the chosen root adopted the root folder's own name as its class.
- **The prompt comes from the dataset.** A deliberate failure run proved it: with the built-in seven-label prompt and a custom two-class dataset, the result was 0% accuracy and 100% invalid output.  
`buildVisionPrompt(labels)` builds the closed label set from whatever the user brought. Re-run properly with `red_panel/green_panel`: 100%, passed.
- **The metrics schema had to be widened.** `perClass` was pinned to exactly seven entries, so a different class count ran to completion and *then* failed validation.
- **Four declarations block the run** - SPDX licence text plus three privacy confirmations. A dataset with no stated licence and no confirmation of what is in the images is not one anybody should publish a metric from.

### Files

| File                               | What it is                                                                                                                                                                                                                                                                                                                |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>RunBuiltInDataset.tsx</code> | Runs the shipped 21-image dataset from the page, on Ollama (this computer), Gemini or Groq, and shows the result as a table with the cell that missed its threshold highlighted. Model lists come from each provider; the cloud lists are narrowed to models that accept an image                                         |
| <code>builtInDataset.ts</code>     | The Ollama run, performed IN THE BROWSER: fetches the manifest and images from <code>/api/v1/vision-benchmarks/dataset</code> , runs the shared executor against the visitor's own Ollama, then reads residency for hardware fit. Hosted, the server has no Ollama, so this is the only way a local vision run can happen |
| <code>runHistory.ts</code>         | Every run this browser has made - vision, text and code, and comparisons - each capped at 20. Every access is wrapped, because <code>localStorage</code> throws outright in some privacy modes rather than returning null                                                                                                 |
| <code>ThresholdNote.tsx</code>     | States the gate: accuracy, macro F1, invalid-output and success-rate bars, rendered from <code>DEFAULT_VISION_THRESHOLDS</code> rather than typed in, so the copy cannot drift from the rule the evaluator applies                                                                                                        |
| <code>DatasetUpload.tsx</code>     | 403 lines. Folder-per-class picker, <code>labelOf</code> , the four blocking declarations, prompt construction from the dataset's own labels, and evidence JSON download                                                                                                                                                  |

### Connected folders

- `src/modules/vision-benchmark/infrastructure/` - `browser-image-processor.ts` and `browser-ollama-provider.ts`, the two adapters that exist so this can work in the tab.
- `src/app/vision-benchmark/` - the page that mounts it.
- `src/modules/vision-benchmark/core/` - `buildVisionPrompt` and the widened schemas.
- `datasets/vision-benchmark/` - the reference dataset this generalises away from.

## Referenced from

---

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/vision-benchmarks/`
- `src/app/history/`
- `src/app/api/v1/vision-benchmarks/dataset/`
- `src/components/`
- `src/components/compare/`
- `src/components/history/`
- `src/modules/vision-benchmark/`